TP1 & TD1 : la classe GrayImage et le format PGM

R3.L01 — Représentations et codages des images

Fiche d'explication du TP, écrite à partir du cours (CM diapos 8 à 28), du sujet du TD1 et du corrigé pris en TD.

Code complet et testé : `PROJET_FINAL/Image.hpp`, `Image.cpp`, `tp1.cpp`

Ce qu'il faut rendre

Fonctions demandées (noms et prototypes à respecter exactement)

```
GrayImage(uint16_t w, uint16_t h);              // + copie, destructeur
const uint16_t& getWidth() const;   const uint16_t& getHeight() const;
uint8_t& pixel(uint16_t x, uint16_t y);         // + version const
void clear(uint8_t gray = 0);
void rectangle    (x, y, w, h, gray);           // cadre de 1 pixel
void fillRectangle(x, y, w, h, gray);           // rectangle plein
void writePGM(std::ostream&) const;
static GrayImage* readPGM(std::istream&);
void skip_line(std::istream&);                   // TD1 question 3
void skip_comments(std::istream&);               // TD1 question 4
```

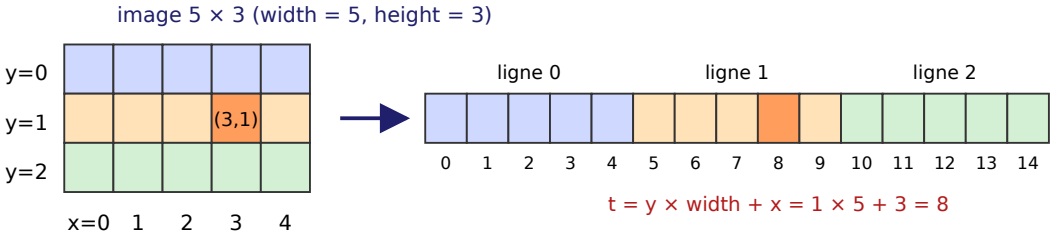
Rappel (note.txt)

- TP noté à la 5e séance : on rend **tout** le code des 5 séances.
- Les noms de fonctions comptent.
- Contrôle de TD : **14/10/2026**.
- Le sujet annonce aussi `writeTGA`, `writeJPEG`... pour GrayImage : faits (TGA 3 et 11, JPEG gris).

Toutes les erreurs se signalent par des **exceptions** de `<stdexcept>` :

- `invalid_argument` : dimension nulle ;
- `out_of_range` : pixel ou rectangle hors de l'image ;
- `runtime_error` : fichier illisible ou mal formé.

Une image en mémoire : le tableau « à plat » CM diapo 25



On met les lignes **bout à bout** dans un seul tableau de `width × height` cases. Le pixel `(x, y)` est à l'indice **`t = y × width + x`** : on saute `y` lignes complètes, puis on avance de `x`.

Pourquoi un seul tableau ? Une seule allocation, un seul `delete[]`, et surtout on peut écrire ou lire **tous les pixels d'un coup** avec `write`/`read` (les pixels sont rangés dans le même ordre que dans le fichier PGM).

La classe GrayImage CM diapos 26 et 28

```
class GrayImage { // N'oubliez pas <cstdint> !
  const uint16_t width, height; // dans [0;65535]
        uint8_t  *array;        // pixels dans [0;255]
  GrayImage() = delete;         // pas d'image sans taille
public:
  GrayImage& operator=(const GrayImage& b) = delete;
  GrayImage(uint16_t w, uint16_t h);
  GrayImage(const GrayImage& orig);
  ~GrayImage();
  inline const uint16_t& getWidth()  const { return width;  }
  inline const uint16_t& getHeight() const { return height; }
  inline uint8_t& pixel(uint16_t x, uint16_t y) {
    if (x >= width || y >= height)
      throw std::out_of_range("GrayImage::pixel : hors de
l'image");
    return array[y * width + x];
  }
  inline const uint8_t& pixel(uint16_t x, uint16_t y) const { /*
idem */ }
  ...
};
```

- `uint8_t` : un pixel = exactement 1 octet, de 0 (noir) à 255 (blanc).
- `const` **sur les dimensions** : la taille ne change jamais, donc on ne réalloue jamais le tableau.
- `= delete` : une image sans taille n'a pas de sens ; et `a = b` changerait la taille de `a`, impossible car `width` est `const`.
- `pixel()` **renvoie une référence** : on peut l'utiliser à gauche d'un `=`, ex. `im.pixel(10,20) = 255;`
- La version `const` sert quand l'image est constante (ex. dans `writePGM()` `const`).

Constructeurs et destructeur CM diapo 27

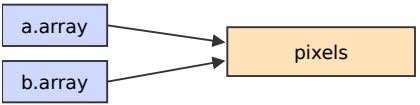
```
GrayImage::GrayImage(uint16_t w, uint16_t h)
  : width(w), height(h), array(nullptr) // liste d'initialisation
{
  if (w == 0 || h == 0)
    throw std::invalid_argument("GrayImage : largeur et hauteur
doivent être > 0");
  array = new uint8_t[size_t(width) * height];
}

GrayImage::GrayImage(const GrayImage& o) // copie PROFONDE
  : width(o.width), height(o.height), array(nullptr)
{
  array = new uint8_t[size_t(width) * height];
  for (size_t t = 0; t < size_t(width) * height; t++)
    array[t] = o.array[t];
}

GrayImage::~GrayImage()
{ delete[] array; }  // new[] -> delete[]
```

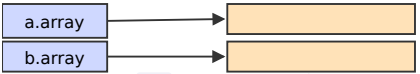
`width` étant `const`, on doit l'initialiser dans la liste après les « : » (impossible de l'affecter dans le corps).

Sans constructeur de copie :



2 objets, 1 tableau : 2 delete[] → plantage

Avec (copie profonde) :

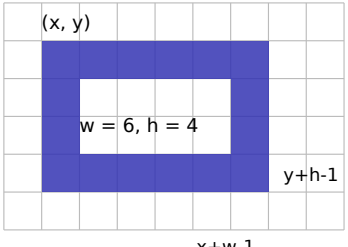


- **Pourquoi `new` ?** On ne connaît la taille qu'à l'exécution, et une image est trop grosse pour la pile (*stack*) : on la met sur le tas (*heap*).
- On n'initialise pas les pixels : `clear()` ou `readPGM()` le feront.
- `size_t(width) * height` : avec deux `uint16_t`, le calcul se fait en `int` et 65535 × 65535 déborde.

clear, rectangle et fillRectangle

```cpp
void GrayImage::clear(uint8_t gray)  // gray = 0 par défaut (dans le .hpp)
{
  for (size_t t = 0; t < size_t(width) * height; t++)
    array[t] = gray;
}

void GrayImage::rectangle(uint16_t x, uint16_t y, uint16_t w, uint16_t h, uint8_t gray)
{
  if (w == 0 || h == 0) return;                    // rien à dessiner
  if (x + w > width || y + h > height)
    throw std::out_of_range("GrayImage::rectangle : le rectangle sort de l'image");
  for (uint16_t i = 0; i < w; i++) {
    pixel(x + i, y)          = gray;   // bord haut
    pixel(x + i, y + h - 1) = gray;   // bord bas
  }
  for (uint16_t j = 0; j < h; j++) {
    pixel(x,         y + j) = gray;   // bord gauche
    pixel(x + w - 1, y + j) = gray;   // bord droit
  }
}
// fillRectangle : même vérification, puis double boucle j (lignes), i (colonnes)
```



- Le cadre occupe les colonnes `x` à `x+w-1` : **w** pixels, pas w+1.
- On vérifie avant de dessiner : on ne laisse pas un rectangle à moitié tracé.
- `x + w` est calculé en `int` : pas de débordement.

Le format PGM « P5 » : l'exemple du TD1, octet par octet

```
00000000  50 35 0a 23 20 4d 61 64 65 20 62 79 20 45 2e 20
00000010  52 65 6d 79 0a 35 20 34 0a 32 35 35 0a 00 ff 00
00000020  ff 00 10 20 40 80 ff 80 40 80 c0 80 80 80 80 80
00000030  80
```

"P5" + LF (magic number)
"# Made by E. Remy" + LF
"5 4" + LF (largeur, hauteur)
"255" + LF (valeur maximale)
5 × 4 = 20 octets de pixels

Total : 29 octets d'en-tête + 20 octets de pixels = 49 = 0x31 octets (la dernière adresse affichée).

- L'**en-tête est du texte ASCII** : `0x50` = 'P', `0x35` = '5', `0x0a` = LF (10), `0x20` = espace, `0x23` = '#'.
- La largeur 5 est écrite avec le **caractère** '5' (`0x35`), pas l'octet 5.
- Les **pixels sont binaires** : un octet par pixel, ligne par ligne, en commençant en haut à gauche.

	x=0	1	2	3	4
y=0	0	255	0	255	0
y=1	16	32	64	128	255
y=2	128	64	128	192	128
y=3	128	128	128	128	128

(Vérifié : notre `readPGM` relit exactement ces valeurs.)

Écrire un PGM : writePGM TD1 question 2

```
void GrayImage::writePGM(std::ostream& os) const
{
  // En-tête en texte ASCII (opérateur <<)...
  os << "P5\n"
     << "# Image sauvegardée par " << AUTEUR << " pour les TP de
RepCod.\n"
     << width << " " << height << "\n"
     << 255 << "\n";
  // ... puis les pixels en binaire, d'un seul bloc (CM diapo 20).
  os.write((const char*)array, size_t(width) * height);
  if (!os)
    throw std::runtime_error("writePGM : erreur d'écriture");
}
```

```
std::ofstream ofs("tp1_rectangles.pgm", std::ios::binary); //
binary !
im.writePGM(ofs);
```

- **`<<` pour l'en-tête** : il écrit le nombre 320 sous forme de texte « 320 » (3 caractères). C'est ce que veut l'en-tête.
- **`write` pour les pixels** : il recopie les octets tels quels. Avec `<<`, un pixel 200 deviendrait le texte « 200 ».
- `write` attend un `const char*` et un nombre d'octets : on convertit le pointeur avec `(const char*)`.
- Le paramètre est un **`std::ostream&`** : la même fonction écrit dans un fichier, dans `std::cout` ou dans un `stringstream`.
- Le fichier doit être ouvert avec **`std::ios::binary`** (CM diapo 9), sinon Windows transforme chaque octet 10 en 13,10.

Lire l'en-tête : skip_line et skip_comments

TD1 questions 3 et 4

```cpp
// Lit des caractères jusqu'au « Line Feed » (inclus).
void skip_line(std::istream& is)
{
  char c;
  while (is.get(c) && c != '\n') {}
}

// Lit un caractère : si c'est '#', on saute la ligne
// et on recommence ; sinon on le remet sur le flux
// avec putback(), comme s'il n'avait jamais été lu.
void skip_comments(std::istream& is)
{
  char c;
  while (is.get(c) && c == '#')
    skip_line(is);
  if (is)
    is.putback(c);
}
```

`is.get(c)` renvoie le flux : il vaut « faux » en fin de fichier, ce qui évite une boucle infinie (le `while (c != '\n')` des notes boucle pour toujours si le LF n'arrive jamais).

Le piège de chat.pgm

L'en-tête de `chat.pgm` contient des commentaires **après** les dimensions :

```
320 240⏎
# Encore du bordel⏎
# après les dimensions⏎
# de l'image.⏎
255⏎
```

`is >> w >> h` s'arrête **juste avant** le ⏎ rouge. Appelée tout de suite, `skip_comments` lit ce ⏎, voit que ce n'est pas '#' et le remet : aucun commentaire sauté, et `is >> maxval` échoue sur '#'.

Solution : `skip_blanks(is)` (`get` + `putback`) *avant* `skip_comments(is)`. Marche aussi avec les fins de ligne Windows (PGM.txt : « *be as lenient as possible* »).

Lire un PGM : readPGM TD1 question 5

```
GrayImage* GrayImage::readPGM(std::istream& is)
{
  std::string magic;
  is >> magic;              // lit "P5"
  if (magic != "P5" && magic != "P2")
    throw std::runtime_error("readPGM : ce n'est pas un
fichier PGM");
  skip_blanks(is);          // saute les blancs (LF,
CR, espaces)
  skip_comments(is);
  unsigned int w = 0, h = 0;
  if (!(is >> w >> h))
    throw std::runtime_error("readPGM : largeur et
hauteur illisibles");
  if (w == 0 || h == 0 || w > 65535 || h > 65535)
    throw std::runtime_error("readPGM : dimensions
invalides");
  skip_blanks(is);          // saute le LF laissé par
>>
  skip_comments(is);
  unsigned int maxval = 0;
  if (!(is >> maxval))
    throw std::runtime_error("readPGM : valeur maximale
illisible");
  if (maxval != 255)
    throw std::runtime_error("readPGM : seul 255 est
géré");
```

```
  GrayImage* res = new GrayImage(uint16_t(w),
uint16_t(h));
  if (magic == "P5") {
    is.get();   // UN SEUL caractère blanc après 255
    is.read((char*)res->array, size_t(w) * h);
  } else {      // "P2" (bonus) : pixels écrits en
décimal
    for (size_t t = 0; t < size_t(w) * h; t++) {
      unsigned int v = 0;
      if (!(is >> v) || v > 255) { delete res; throw
std::runtime_error("..."); }
      res->array[t] = uint8_t(v);
    }
  }
  if (!is) {    // fichier tronqué ou corrompu
    delete res; // on libère avant de signaler l'erreur
    throw std::runtime_error("readPGM : pixels
incomplets ou invalides");
  }
  return res;
}
```

- On lit `w` et `h` dans des `unsigned int` pour pouvoir **vérifier** qu'ils tiennent dans un `uint16_t`.

readPGM : les deux questions à savoir expliquer

Pourquoi readPGM est static ?

Une méthode normale s'appelle **sur une image qui existe déjà** (`im.writePGM(f)`). Or c'est justement `readPGM` qui fabrique l'image : il faudrait une image pour créer une image, « le serpent qui se mord la queue » (TD1).

Avec `static`, la fonction est attachée à la **classe** :

```
std::ifstream f("images/chat.pgm",
std::ios::binary);
GrayImage* im = GrayImage::readPGM(f);  // pas
d'objet avant
...
delete im;  // c'est l'appelant qui libère
```

Elle reste membre de `GrayImage` : elle a le droit d'écrire dans `res->array`, qui est privé.

Pourquoi un seul is.get() après 255 ?

PGM.txt : « *only a single character of whitespace (typically a newline) is allowed after the maxval* ».



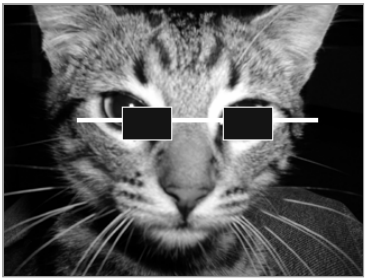
Pixel 10 = code de LF, pixel 32 = code de l'espace !
Si on sautait *tous* les blancs (avec `>>` ou un `skip_comments` qui saute les espaces), les pixels 10 et 32 seraient avalés : l'image serait décalée et il manquerait des octets à la fin. Avec `chat.pgm`, ça arrive vraiment (des pixels valent 10).

Tester : le chat à lunettes

```
std::ifstream ifs("images/chat.pgm", std::ios::binary);
GrayImage* chat = GrayImage::readPGM(ifs);
chat->fillRectangle(65, 100, 215, 4, 255); // la barre
chat->fillRectangle(105, 90, 45, 30, 20);  // verre gauche
chat->fillRectangle(195, 90, 45, 30, 20);  // verre droit
chat->rectangle(105, 90, 45, 30, 255);     // monture
chat->rectangle(195, 90, 45, 30, 255);
std::ofstream ofs("tp1_chat_lunettes.pgm", std::ios::binary);
chat->writePGM(ofs);
delete chat;
```

Le fichier `chat_lunettes.pgm` fourni a été comparé pixel par pixel avec notre résultat : **0 pixel différent**. Les coordonnées ci-dessus sont donc exactement celles de l'image de référence.

```
cd PROJET_FINAL
make tp1 && ./tp1
eog images/chat.pgm tp1_chat_lunettes.pgm
```



Le programme teste aussi : une image non carrée 400 × 250, la lecture d'un PGM « P2 » (bonus), et deux exceptions (pixel hors de l'image, fichier qui n'est pas un PGM).

Ce qui a été corrigé par rapport à la version de Gemini

Point	Version Gemini	Correction
`skip_comments`	Sautait aussi les espaces et les LF : ça marche, mais ce n'est pas la fonction demandée au TD1 (question 4).	Version du TD (lit, teste '#', `putback`), précédée de `skip_blanks`.
Commentaire écrit	« Image sauvegardee pour les TP de RepCod. » sans nom.	« Image sauvegardée par *nom* pour les TP de RepCod. » (demandé au TD).
Taille du tableau	`width * height` calculé en `int`.	`size_t(width) * height`.
Lecture P2	Valeurs > 255 acceptées sans erreur.	Valeur > 255 = fichier invalide.
Test	Rectangles au hasard.	Reproduit l'image fournie `chat_lunettes.pgm` et le vérifie.
Organisation	`GrayImage.hpp/.cpp` à part, abandonnés dès le TP2.	Tout dans `Image.hpp/.cpp` (demandé au TP2), rendu unique.
PDF	Pied de page « E. Remy », citations du prof inventées.	Fiche de révision présentée comme telle.

Questions possibles au contrôle

1. **Où est le pixel (x, y) dans `array` ?** À l'indice `y × width + x`.
2. **Taille du fichier PGM d'une image 320 × 240 ?** En-tête (quelques dizaines d'octets) + 320 × 240 = 76 800 octets de pixels.
3. **Pourquoi `ios::binary` ?** Pour éviter les conversions de fin de ligne (CM diapo 9) : sinon des octets 10 ou 13 peuvent être modifiés.
4. **Pourquoi `write` et pas `<<` pour les pixels ?** `<<` écrit du texte (« 200 » = 3 octets), `write` écrit l'octet 200.
5. **Pourquoi `readPGM` est `static` ?** Elle crée l'image : on ne peut pas exiger qu'une image existe déjà pour l'appeler.
6. **Que fait `putback(c)` ?** Il remet le caractère `c` dans le flux : la prochaine lecture le relira.
7. **Pourquoi interdire `operator=` ?** Les dimensions sont `const` : `a = b` ne peut pas changer la taille de `a`.
8. **Pourquoi un constructeur de copie ?** La copie par défaut copierait le pointeur : deux images partageraient le même tableau, puis deux `delete[]`.
9. **Après 255, combien de blancs ?** Exactement un (PGM.txt). Le premier pixel peut lui-même valoir 10 ou 32.